

Étude de différents algorithmes servant à la résolution du problème de coloration de graphes

Intriguée par l'optimisation des ressources en informatique, j'ai cherché à gérer via la coloration de graphes des conflits d'accès simultanés comme les créneaux d'emplois du temps. L'étude d'algorithmes heuristiques pour palier la complexité de ce problème m'a particulièrement intéressée.

La notion de cycle est ancrée dans l'étude des graphes. Un graphe sans cycle admet une 2-coloration en temps linéaire. C'est la présence de cycles qui rend le problème difficile.

Le candidat atteste avoir travaillé en groupe.

Liste des membres du groupe :

- AUDREN Tudwal
- SUARD Philippe

Positionnement thématique (ÉTAPE 1) :

- INFORMATIQUE (*Informatique pratique*)
- INFORMATIQUE (*Informatique Théorique*)
- MATHÉMATIQUES (*Mathématiques Appliquées*)

Mots-clés (ÉTAPE 1) :

Mots-clés (en français) Mots-clés (en anglais)

<i>Graphe</i>	<i>Graph</i>
<i>Nombre chromatique</i>	<i>Chromatic number</i>
<i>Degré de saturation</i>	<i>Degree of saturation</i>
<i>Problème NP</i>	<i>NP problem</i>
<i>Complexité</i>	<i>Complexity</i>

Bibliographie commentée

En informatique, il est essentiel d'évaluer et de minimiser la complexité des programmes. Cela permet d'estimer l'ordre de grandeur du temps d'exécution d'un programme en fonction de la taille de ses données d'entrée. Au delà de la complexité des programmes, on peut s'intéresser à celle d'un problème. Un problème est d'une classe de complexité donnée s'il existe un algorithme le résolvant en cette complexité. En particulier, les problèmes de décision, c'est-à-dire les problèmes pour lesquels la réponse est "oui" ou "non", peuvent être classés selon ce critère.

Un problème de décision appartient à la classe NP (Non-deterministic Polynomial time) si, étant donnée une solution candidate, il est possible d'en vérifier la validité en temps polynomial [1]. Le problème SAT est un exemple classique de problème NP. Étant donnée une formule logique en forme normale conjonctive, SAT consiste à déterminer s'il existe une affectation de valeurs booléennes rendant la formule vraie [2]. Le problème SAT est non seulement dans NP, mais il est également NP-complet, c'est-à-dire qu'il est au moins aussi difficile que tout autre problème de la classe NP. La notion de NP-complétude a été introduite en 1971 par Stephen Cook, prouvant dès cette publication la NP-complétude du problème SAT [3]. À la suite de ce travail, de nombreux problèmes de décision ont été classés NP-complets.

En 1972, Richard Karp a montré la NP-complétude de 21 problèmes fondamentaux de l'informatique théorique par réductions successives depuis SAT [4]. Parmi ces problèmes figurent plusieurs problèmes liés à la théorie des graphes, tels que le problème de la clique, qui consiste à décider s'il existe un sous-graphe complet de taille donnée dans un graphe [5], ou encore le problème du chemin hamiltonien qui consiste, sur un graphe fixé, à décider s'il admet un chemin passant une fois et une seule sur chaque sommet dudit graphe. Un autre de ces problèmes est celui de la k-coloration de graphes. Étant donné un graphe simple, c'est-à-dire sans arête reliant un sommet à lui-même, le problème de coloration consiste à déterminer s'il est possible de colorer les sommets à l'aide d'un nombre fixé de couleurs de sorte que deux sommets adjacents n'aient jamais la même couleur. La coloration de graphes permet de modéliser de nombreuses situations concrètes d'optimisation, dont les problèmes de décision peuvent être résolus à l'aide de certains algorithmes relatifs à la coloration de graphes comme EPCOT [6]. C'est notamment le cas de la coloration de cartes en un minimum de couleurs où deux régions limitrophes ne peuvent avoir la même couleur, de l'organisation d'emplois du temps afin d'éviter les conflits horaires, ou encore de l'attribution de fréquences radio à des émetteurs géographiquement proches pour limiter les interférences.

Dans la plupart des contextes d'application, il est nécessaire de construire une solution et pas seulement d'en décider l'existence. Se pose alors la question de l'optimalité du nombre de couleurs fixé en entrée. On appelle nombre chromatique le plus petit entier k pour lequel un graphe est colorable avec k couleurs. L'intérêt est de déterminer ce nombre, car en plus de fournir une solution, on cherche à construire cette dernière avec le minimum de couleurs possible. Des algorithmes exacts, tels que l'algorithme de Zykov ou des approches par backtracking [7], permettent de calculer le nombre chromatique et de construire une solution optimale.

Lorsque la taille des graphes devient importante, typiquement au-delà de 80 sommets, le temps de calcul devient prohibitif en raison de la NP-complétude du problème. Il est donc nécessaire de recourir à des algorithmes approchés, qui sacrifient l'optimalité de la solution au profit de la rapidité d'exécution. Parmi les heuristiques les plus couramment utilisées figurent l'algorithme First-Fit et son amélioration DSATUR [7]. Toutefois, pour se rapprocher davantage d'une solution optimale, il est essentiel d'étudier les propriétés mathématiques des graphes afin de concevoir des heuristiques plus pertinentes et plus efficaces [8].

Problématique retenue

Quels sont les algorithmes les plus performants pour trouver une coloration optimale ou approchée et par quels moyens les comparer ?

Objectifs du TIPE du candidat

- Comprendre l'intérêt du problème de la coloration de graphes en vue d'applications concrètes
- Implémenter un algorithme naïf puis un algorithme exact par backtracking afin d'évaluer la difficulté à construire une solution optimale
- Implémenter des algorithmes heuristiques tels que DSATUR et les comparer suivant leur complexité et la précision de la coloration obtenue par rapport à une coloration optimale
- Dégager des structures de graphes pour lesquelles certains algorithmes sont plus efficaces

Références bibliographiques (ÉTAPE 1)

- [1] GUILLAUME LAGARDE : La théorie de la complexité algorithmique pour calculer efficacement : <https://interstices.info/la-theorie-de-la-complexite-algorithmique/>
- [2] DENIS TRYSTRAM : Le problème SAT et ses variantes : <https://datamove.imag.fr/denis.trystram/SupportsDeCours/lesson5SAT.pdf>
- [3] STEPHEN COOK : The complexity of theorem-proving procedures : Association for Computing Machinery : ISBN 9781450374644, DOI 10.1145/800157.805047
- [4] RICHARD KARP : Complexity of Computer Computations : Reducibility among Combinatorial Problems. : *Editors Miller, R.E., Thatcher, J.W., Bohlinger, J.D. (1972), ISBN 978-1-4684-2003-6*
- [5] JOCELYN BERNARD, HAMIDA SEBA : Résolution de problèmes de cliques dans les grands graphes. : *[Rapport de recherche] LIRIS. 2015. hal-01284640*
- [6] DOMINIQUE DE WERRA, DANIEL KOBLER : Coloration de graphes : Fondements et applications : *RAIRO Oper. Res. 37 (2003) 29-66 DOI: 10.1051/ro:2003013*

[7] OLIVIER TOGNI : L3: cours Graphes I6S3 Chapitre III : coloration de graphe : <http://o.togni.u-bourgogne.fr/CMGraphesCh3.pdf>

[8] ÉRIC SOPENA : Homomorphismes et Colorations de Graphes : https://www.labri.fr/perso/sopena/pmwiki/uploads/Site/Cours_Homomorphismes%20et%20Colorations%20de%20Graphes_M2.pdf

DOT

[1] : [Septembre 2025] Choix du sujet de TIPE sur la coloration de graphes, recherches bibliographiques sur le sujet et début d'implémentations naïves.

[2] : [Octobre 2025] Implémentation des algorithmes First-Fit, Welsh-Powell et DSATUR en OCaml. Difficultés rencontrées pour minimiser la complexité de l'algorithme DSATUR. Implémentation de fonctions génératrices de graphes aléatoires en vue de comparer les différents algorithmes.

[3] : [Novembre 2025] Utilisation de Graphviz pour représenter des graphes. La position automatisée des sommets rendant peu clairs les résultats, ce logiciel sera abandonné au profit de Tikz. Implémentation de l'algorithme Branch-and-Bound pour obtenir des colorations exactes de nos graphes. Premières comparaisons entre les différents algorithmes.

[4] : [Décembre 2025] Réalisation d'une première base de données comportant plusieurs milliers de graphes de tailles et degrés moyens variables. Recherches infructueuses pour trouver comment exploiter ces bases de données en appliquant les différents algorithmes heuristiques sur chacun des graphes.

[5] : [Début mars 2026] Constitution d'une seconde base de données plus fournie. Exécution des différents algorithmes sur ces graphes et stockage de leurs performances sous tableur. Pour des raisons de complexité, nous avons dû nous restreindre à exécuter Branch-and-Bound sur des graphes de taille inférieure à 20.

[6] : [Fin mars 2026] Premières courbes expérimentales comparant les temps d'exécution des programmes ainsi que les colorations renvoyées. Leur complexité n'étant pas celle recherchée, nous avons corrigé nos programmes afin qu'ils soient de complexité souhaitée.

[7] : [Mai 2026] Sélection des courbes expérimentales pertinentes pour la présentation, analyse des complexités corrigées ainsi que des colorations renvoyées.